

FairReward 50/50

Your message

fairreward_frontend1.html

fairreward_frontend.html

TruffaAppBingo_SoldiePubblicita3-GoogleGemini.pdf

TruffaAppBingo_SoldiePubblicita-2-GoogleGemini.pdf

TruffaAppBingo_SoldiePubblicita-GoogleGemini.pdf

sql: CREATE EXTENSION IF NOT EXISTS "pgcrypto";

```
CREATE TABLE IF NOT EXISTS users ( id UUID PRIMARY KEY
DEFAULT gen_random_uuid(), email TEXT UNIQUE NOT NULL,
balance NUMERIC(14,2) NOT NULL DEFAULT 0 );
```

```
CREATE TABLE IF NOT EXISTS tasks ( id UUID PRIMARY KEY
DEFAULT gen_random_uuid(), title TEXT NOT NULL, required_quota
NUMERIC(14,2) NOT NULL, total_reward NUMERIC(14,2) NOT NULL,
cofinancing_allowed BOOLEAN DEFAULT false );
```

```
CREATE TABLE IF NOT EXISTS fr_liquidity ( id INT PRIMARY KEY
DEFAULT 1, available_amount NUMERIC(18,2) NOT NULL DEFAULT
0 );
```

```
INSERT INTO fr_liquidity(id,available_amount) VALUES(1,3000.00)
ON CONFLICT(id) DO UPDATE SET
available_amount=EXCLUDED.available_amount; Controlla se hai
capito bene cosa serve questa applicazione e fammi un check di
quello che hai trovato e se vuoi di quello che manca e così almeno
riesco a darti tutto prima di partire a lavorare
```

Show more

4 hours ago

Base44 response

Read the web

Base44 response

Ottimo, ho analizzato tutto il materiale. Ecco il mio check completo:

✓ Ho capito — FairReward 50/50

Cosa è l'app

Una piattaforma dove gli utenti **completano micro-task** (es. "scarica un'app", "registrati su un sito") e ricevono una **ricompensa in denaro**. La logica "50/50" significa che il premio viene cofinanziato: metà dalla liquidità FairReward (il pool), metà dalla quota che l'utente stesso "investe" dal proprio saldo interno.

Entità dati che ho identificato (dal SQL + frontend)

Tabella	Cosa fa
users	Utenti con email + saldo interno (€)
tasks	I lavori disponibili: titolo, quota richiesta, ricompensa totale, flag cofinanziamento
fr_liquid ity	Pool di liquidità FairReward (parte iniziale €3.000)

Mancante dal SQL (ma implicito dalla logica):

- ✗ participations — serve una tabella che tracci CHI ha partecipato a QUALE task, quando, e con che stato (pending / completed / paid). Senza questa non si può gestire l'idempotency-key né impedire doppie partecipazioni.

Frontend (i due HTML sono identici)

Mostra:

- **Saldo utente** (€0,00)
- **Catalogo task** con quota richiesta e ricompensa totale
- Pulsanti "Inizia ora" e "Dettagli"

- Sezione "lavori attivi" (vuota)

Mancante nel frontend:

- ❌ Nessuna autenticazione/login utente (i placeholder `USER_ID` / `AUTH_TOKEN` sono hardcoded)
- ❌ Nessuna connessione reale al backend (l'URL di Netlify è un placeholder)
- ❌ Nessuna schermata di conferma pagamento / esito partecipazione
- ❌ Nessuna storico transazioni/partecipazioni per l'utente

Backend (Netlify Function `participate.js`)

Logica attesa (dall'idempotency-key e dai parametri):

- 1 Riceve `user_id`, `task_id`, `request_cofinancing`, `Idempotency-Key`
- 2 Verifica saldo utente $\geq$ quota richiesta
- 3 Verifica liquidità `FairReward` sufficiente
- 4 Scala la quota dal saldo utente
- 5 Aggiunge la parte `FairReward`
- 6 Accredita la ricompensa totale
- 7 Restituisce il risultato

Non ho ricevuto il file `participate.js` — ce l'hai? È il pezzo più critico.

Cosa mi serve per partire

Per costruire tutto su Base44 ho bisogno di confermare:

- 1 **`participate.js`** — la logica del Netlify function (il cuore della app)
- 2 Conferma se vuoi che io gestisca anche la **tabella `participations`** (la aggiungerei)
- 3 Vuoi autenticazione email/password per gli utenti, o per ora lavoriamo con un unico utente admin?

Dimmi cosa manca e partiamo! 🚀

4 hours ago

Your message

index.html html

```
<!doctype html> <html lang="it"> <head> <meta charset="utf-8" />
<meta name="viewport" content="width=device-width,initial-
scale=1" /> <title>FairReward 50/50</title> <style> :root{--
primary:#0EA5A4;--accent:#22C55E;--bg:#071021;--
card:#0B1220;--muted:#9AA6B2}
html,body{height:100%;margin:0;background:linear-
gradient(180deg,var(--bg),#0f1720);color:#e6eef6;font-
family:Inter,system-ui,Arial} .app{max-width:420px;margin:0
auto;padding:18px} .header{display:flex;align-items:center;justify-
content:space-between;margin-bottom:12px} .logo{display:flex;align-
items:center;gap:10px} .logo svg{width:44px;height:44px;border-
radius:10px} .balance{background:var(--card);padding:14px;border-
radius:12px;margin-bottom:12px;box-shadow:0 6px 18px
rgba(2,6,23,0.6)} .balance .amount{font-size:20px;font-
weight:700;color:var(--accent)} .section-title{font-
size:13px;color:var(--muted);margin:12px 0 8px} .task-
card{background:rgba(255,255,255,0.03);padding:12px;border-
radius:12px;margin-bottom:10px;display:flex;justify-content:space-
between;align-items:center} .task-left{max-width:65%} .task-
title{font-weight:600;margin-bottom:6px} .task-meta{font-
size:12px;color:var(--muted)} .btn{background:var(--
accent);color:#042018;padding:8px 12px;border-
radius:10px;border:0;font-
weight:700} .btn.ghost{background:transparent;border:1px solid
rgba(255,255,255,0.06);color:var(--
muted)} .modal{position:fixed;inset:0;display:none;align-
items:center;justify-
content:center;background:rgba(2,6,23,0.6)} .modal.show{display:fle
x} .modal-card{background:#071021;padding:16px;border-
radius:14px;width:92%;max-width:420px} .muted{color:var(--
muted);font-
size:13px} .footer{position:fixed;left:0;right:0;bottom:0;padding:10px;
background:linear-
gradient(180deg,transparent,rgba(0,0,0,0.6));display:flex;justify-
content:center} .small{font-size:12px;color:var(--muted)} </style> </
head> <body> <div class="app"> <div class="header"> <div
class="logo"> <!-- inline logo --> <svg xmlns="http://www.w3.org/
```

```

2000/svg" width="44" height="44" viewBox="0 0 120 120" aria-
hidden="true"> <rect rx="20" width="120" height="120"
fill="#0EA5A4"/> <g transform="translate(18,18)" fill="#071021">
<path d="M12 6h6v36h-6zM30 18h6v24h-6zM48 0h6v42h-6z" />
</g> </svg> <div> <div style="font-weight:700">FairReward</div>
<div class="small">Trasparenza 50/50 • Milano</div> </div> </div>
<div class="small">Saldo reale</div> </div> <div class="balance"
id="balanceCard"> <div class="small">IL TUO SALDO</div> <div
class="amount" id="balanceAmount">€0,00</div> <div
class="small" style="margin-top:8px">Solo saldo FairReward per
partecipare. Nessun versamento esterno.</div> </div> <div
class="section-title">Catalogo lavori</div> <div class="task-card"
data-task-id="__TASK_ID__" data-required-quota="10.00" data-
cofinancing="true"> <div class="task-left"> <div class="task-
title">Download & Registrazione Base</div> <div class="task-
meta">Tempo 3 min • Ricompensa totale €24 • Quota richiesta €10</
div> </div> <div> <button class="btn"
onclick="startTask(this)">Inizia ora</button> <div
style="height:6px"></div> <button class="btn ghost"
onclick="showDetails(this)">Dettagli</button> </div> </div> <div
id="activeList" class="section-title small">Nessun lavoro attivo</div>
</div> <div id="modal" class="modal" role="dialog" aria-
hidden="true"> <div class="modal-card"> <div
style="display:flex;justify-content:space-between;align-
items:center"> <strong>Conferma partecipazione</strong> <button
onclick="closeModal()" class="btn ghost">Chiudi</button> </div> <p
class="small" id="modalText">Riepilogo...</p> <label
style="display:flex;align-items:center;gap:8px;margin-top:8px">
<input type="checkbox" id="acceptCheckbox" /> <span
class="small">Accetto il prelievo dal saldo FairReward</span> </
label> <div style="display:flex;gap:8px;justify-content:flex-
end;margin-top:12px"> <button class="btn ghost"
onclick="closeModal()">Annulla</button> <button class="btn"
id="confirmBtn" onclick="confirmParticipate()">Partecipa</button>
</div> </div> </div> <div class="footer"> <div style="max-
width:420px;width:100%;display:flex;gap:8px;justify-content:space-
around"> <button class="btn ghost">Home</button> <button
class="btn ghost">Il mio lavoro</button> <button class="btn
ghost">Cassa</button> <button class="btn ghost">Profilo</button>
</div> </div> <script> /* CONFIG: lascia i placeholder così e
sostituisci dopo il deploy se vuoi */ const API_BASE = "https://fair-

```

```

reward.netlify.app/.netlify/functions/participate"; const AUTH_TOKEN
= "__AUTH_TOKEN__"; let user = { id: "__USER_ID__", balance:
12.50 }; function fmt(e){ return '€' + Number(e).toFixed(2); }
document.getElementById('balanceAmount').textContent =
fmt(user.balance); function startTask(btn){ const card =
btn.closest('.task-card'); const required =
parseFloat(card.dataset.requiredQuota || card.getAttribute('data-
required-quota')); const cofin = (card.dataset.cofinancing === 'true'
|| card.getAttribute('data-cofinancing') === 'true'); const userQuota =
user.balance >= required ? required : (cofin ? (required/2) : null);
if(userQuota === null){ alert('Saldo insufficiente. Completa task da 2
minuti per accumulare saldo.');
```

return; } const frPart = (userQuota === required) ? 0 : (required - userQuota);

document.getElementById('modalText').textContent = `Quota richiesta €\${required.toFixed(2)}. Verranno prelevati dal tuo saldo €\${userQuota.toFixed(2)}. FairReward coprirà €\${frPart.toFixed(2)}.`;

document.getElementById('modal').classList.add('show');

window._currentTask = { id: card.dataset.taskId || "__TASK_ID__", userQuota, frPart };

document.getElementById('acceptCheckbox').checked = false; }

function closeModal()

{ document.getElementById('modal').classList.remove('show'); }

async function confirmParticipate(){ const t = window._currentTask;

if(!t) return; if(!

document.getElementById('acceptCheckbox').checked){ alert('Devi accettare il prelievo per procedere.');

return; } const idempotency = crypto.randomUUID();

document.getElementById('confirmBtn').disabled = true; try{ const res = await fetch(API_BASE, { method:'POST', headers:{ 'Content-Type':'application/json', 'Authorization':'Bearer '+AUTH_TOKEN, 'Idempotency-Key': idempotency }, body: JSON.stringify({ user_id: user.id, request_cofinancing: true, task_id: t.id }) }); const json = await res.json(); if(json.status === 'success'){ user.balance = Math.max(0, user.balance - Number(json.data.user_balance_deducted || 0)); document.getElementById('balanceAmount').textContent = fmt(user.balance); alert('Partecipazione avviata'); closeModal(); } else { alert(json.message || json.error_code || 'Errore'); } } catch(e) { alert('Errore di rete'); }

document.getElementById('confirmBtn').disabled = false; } </script> </body> </html> netlify/functions/participate.js js const fetch

```

= require('node-fetch');
exports.handler = async (event) => { try { const SUPABASE_URL =
process.env.SUPABASE_URL || "https://
pzmppfkrqdkxggmzzvs.supabase.co"; const
SUPABASE_SERVICE_KEY = process.env.SUPABASE_SERVICE_KEY
|| "SUPABASE_SERVICE_KEY"; // set as Netlify env

// Basic auth header validation (server must validate token in production)
const authHeader = (event.headers && (event.headers.authorization ||
event.headers.Authorization)) || '';
if (!authHeader.startsWith('Bearer ')) {
  return { statusCode: 401, body: JSON.stringify({ status: 'error',
error_code: 'UNAUTHORIZED', message: 'Missing Bearer token' }) };
}

const body = event.body ? JSON.parse(event.body) : {};
const idempotency = event.headers['idempotency-key'] ||
body.p_idempotency_key || require('crypto').randomUUID();

// Call DB RPC
const rpcBody = {
  p_user_id: body.user_id,
  p_task_id: body.task_id,
  p_request_cofinancing: body.request_cofinancing || false,
  p_idempotency_key: idempotency
};

const res = await fetch(`${SUPABASE_URL}/rest/v1/rpc/participate_in_task`,
{
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'apikey': SUPABASE_SERVICE_KEY,
    'Authorization': `Bearer ${SUPABASE_SERVICE_KEY}`
  },
  body: JSON.stringify(rpcBody)
});

const json = await res.json();

const status = (json && json.status === 'success') ? 200
: (json && json.error_code === 'INSUFFICIENT_FUNDS') ? 402
: (json && json.error_code === 'FR_INSUFFICIENT_LIQUIDITY') ? 409
: (json && (json.error_code === 'USER_NOT_FOUND' || json.error_code ===
'TASK_NOT_FOUND')) ? 404
: 500;

return { statusCode: status, body: JSON.stringify(json) };
} catch (err) { return { statusCode: 500, body: JSON.stringify({ status:
'error', error_code: 'INTERNAL_ERROR', message:
err.message }) }; }; netlify/functions/package.json json { "name":
"fairreward-functions", "version": "1.0.0", "dependencies": { "node-
fetch": "^2.6.7" } } supabase/migration.sql sql CREATE EXTENSION

```



```
IF NOT EXISTS "pgcrypto";
```

```
CREATE TABLE IF NOT EXISTS users ( id UUID PRIMARY KEY  
DEFAULT gen_random_uuid(), email TEXT UNIQUE NOT NULL,  
balance NUMERIC(14,2) NOT NULL DEFAULT 0, level TEXT,  
kyc_status TEXT, created_at TIMESTAMP WITH TIME ZONE  
DEFAULT now() );
```

```
CREATE TABLE IF NOT EXISTS tasks ( id UUID PRIMARY KEY  
DEFAULT gen_random_uuid(), title TEXT NOT NULL, required_quota  
NUMERIC(14,2) NOT NULL DEFAULT 0, total_reward NUMERIC(14,2)  
NOT NULL DEFAULT 0, cofinancing_allowed BOOLEAN DEFAULT  
false, partner_id UUID, created_at TIMESTAMP WITH TIME ZONE  
DEFAULT now() );
```

```
CREATE TABLE IF NOT EXISTS user_tasks ( id UUID PRIMARY KEY  
DEFAULT gen_random_uuid(), user_id UUID NOT NULL  
REFERENCES users(id), task_id UUID NOT NULL REFERENCES  
tasks(id), status TEXT NOT NULL, user_quota NUMERIC(14,2)  
DEFAULT 0, fr_quota NUMERIC(14,2) DEFAULT 0, idempotency_key  
TEXT, created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),  
updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() );
```

```
CREATE UNIQUE INDEX IF NOT EXISTS ux_user_tasks_idempotency  
ON user_tasks(idempotency_key) WHERE idempotency_key IS NOT  
NULL;
```

```
CREATE TABLE IF NOT EXISTS cofinancing ( id UUID PRIMARY KEY  
DEFAULT gen_random_uuid(), user_task_id UUID NOT NULL  
REFERENCES user_tasks(id), fr_amount NUMERIC(14,2) DEFAULT 0,  
user_amount NUMERIC(14,2) DEFAULT 0, status TEXT DEFAULT  
'pending', created_at TIMESTAMP WITH TIME ZONE DEFAULT  
now() );
```

```
CREATE TABLE IF NOT EXISTS transactions ( id UUID PRIMARY KEY  
DEFAULT gen_random_uuid(), user_id UUID REFERENCES users(id),  
amount NUMERIC(14,2) NOT NULL, type TEXT NOT NULL,  
reference_id UUID, status TEXT DEFAULT 'pending', created_at  
TIMESTAMP WITH TIME ZONE DEFAULT now() );
```

```
CREATE TABLE IF NOT EXISTS fr_liquidity ( id INT PRIMARY KEY  
DEFAULT 1, available_amount NUMERIC(18,2) NOT NULL DEFAULT  
0, updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() );
```

```
CREATE TABLE IF NOT EXISTS audit_logs ( id UUID PRIMARY KEY
```



```
DEFAULT gen_random_uuid(), actor TEXT, action TEXT, payload  
JSONB, created_at TIMESTAMP WITH TIME ZONE DEFAULT now() );
```

```
INSERT INTO fr_liquidity(id, available_amount) VALUES (1, 3000.00)  
ON CONFLICT (id) DO UPDATE SET available_amount =  
EXCLUDED.available_amount; README.md md
```

FairReward 50/50

Contenuto:

- index.html
- netlify/functions/participate.js
- netlify/functions/package.json
- supabase/migration.sql
- deploy_instructions.txt
- .gitignore

Placeholders da sostituire:

- **NETLIFY_FUNCTION_URL** (client)
- **SUPABASE_URL** (server; already set to project URL)
- **SUPABASE_SERVICE_KEY** (set as Netlify env)
- **TASK_ID**
- **USER_ID**
- **AUTH_TOKEN**

Non inserire la service_role key nei file client. Impostala come env su Netlify. deploy_instructions.txt txt

- 1 Esegui supabase/migration.sql in Supabase SQL Editor.
- 2 Seed demo user/task (SQL Editor): INSERT INTO users(email, balance) VALUES ('tester@example.com', 10.00) RETURNING id; INSERT INTO tasks(title, required_quota, total_reward, cofinancing_allowed) VALUES ('Download Base', 10.00, 24.00, true) RETURNING id; -> copia gli id restituiti come USER_ID e TASK_ID
- 3 Push repo su GitHub e collega Netlify.
- 4 In Netlify Site settings -> Environment, aggiungi:
SUPABASE_URL = https://pzmppfkrqdkxggmzzvs.supabase.co

SUPABASE_SERVICE_KEY = <service_role_key> <-- set here, never in client files

5 Se vuoi hardcodeare i valori client, sostituisci in index.html: **NETLIFY_FUNCTION_URL** (client endpoint) **AUTH_TOKEN** (test token) **USER_ID TASK_ID**

6 Test endpoint: curl -X POST "https://fair-reward.netlify.app/.netlify/functions/participate" -H "Content-Type: application/json" -H "Idempotency-Key: test-1" -H "Authorization: Bearer <client-test-token>" -d '{"user_id":"<USER_ID>","request_cofinancing":true,"task_id":"<TASK_ID>"}

7 Verifiche DB: SELECT * FROM user_tasks ORDER BY created_at DESC LIMIT 5; SELECT * FROM transactions ORDER BY created_at DESC LIMIT 10; SELECT * FROM fr_liquidity; .gitignore Codice node_modules/ .env .DS_Store

Show more

4 hours ago

Editing isn't available while the app is building

Revert

Reverting isn't available while the app is building

Base44 response

Thought for 8s

Perfetto, ora ho tutto il quadro completo. Ecco il mio check finale prima di partire.



Cosa ho capito — FairReward 50/50

Flusso utente:

- 1 L'utente vede il proprio saldo interno FairReward
- 2 Sceglie un task dal catalogo → vede quota richiesta e ricompensa totale
- 3 Conferma partecipazione (con checkbox di accettazione)

prelievo)

4 Il sistema: preleva la quota dal saldo utente + integra con liquidità FairReward → accredita la ricompensa

5 L'idempotency-key previene doppie partecipazioni

Logica 50/50: se il saldo utente copre tutta la quota → 0 cofinanziamento. Se copre metà → FairReward integra l'altra metà (solo se `cofinancing_allowed`).

 Tabelle identificate (8 totali)

Tabella	Stato	Su Base44
users	✓	→ entità User (built-in)
tasks	✓	→ entità Task
user_tasks	✓	→ entità UserTask (le partecipazioni)
cofinancing	✓	→ entità Cofinancing
transactions	✓	→ entità Transaction
fr_liquidity	✓	→ entità FRLiquidity (riga singola)
audit_logs	✓	→ entità AuditLog
participate_in_task (RPC)	✗ manca	→ backend function Base44

⚠ Nota critica: il `participate.js` (Netlify function) chiama un'RPC Supabase `participate_in_task` che **non è inclusa** nel `migration.sql`. È la stored procedure con tutta la logica transazionale (controllo saldo, scala quota, accredita ricompensa, scrive audit). Serve che mi scrivi anche quella — o me la scrivi tu, o la implemento io come backend function Base44.

 Cosa manca / da chiarire

1 **La RPC `participate_in_task` (SQL)** — il cuore transazionale. Non c'è nei file. La scrivo io come backend function Base44? (*consigliato*)

2 Come vuole ospitare i dati? Base44 ha già un database integrato (entità). Vuoi:

- **(a)** Usare le entità Base44 + una backend function che replica la logica dell'RPC (più semplice, tutto in-platform)
- **(b)** O mantenere Supabase e connetterlo esternamente?

3 Autenticazione utenti: Base44 ha già login/registrazione integrati. Li usiamo invece dei placeholder `__USER_ID__` / `__AUTH_TOKEN__`?

4 Seed dati demo: un utente tester (saldo €10) + un task demo (quota €10, reward €24, cofinancing true). Lo creo?

5 Schermate frontend da fare su Base44:

- Home (saldo + catalogo task) ← dall'HTML che mi hai dato
- "Il mio lavoro" (storico `user_tasks`)
- "Cassa" (transazioni)
- "Profilo"
- Modale conferma partecipazione



Il mio piano proposto

- 1 Creo le 7 entità (Task, UserTask, Cofinancing, Transaction, FRLiquidity, AuditLog + User già esiste)
- 2 Creo la backend function `participateInTask` (replica l'RPC: controllo saldo → scala → accredita → audit, tutto atomico)
- 3 Costruisco la Home come quella dell'HTML, ma concretamente collegata ai dati
- 4 Aggiungo le 3 schermate secondarie (Il mio lavoro / Cassa / Profilo)
- 5 Uso l'autenticazione Base44 al posto dei placeholder
- 6 Faccio il seed dei dati demo

Per partire ho bisogno solo di conferme su punti 1, 2 e 3. Il resto lo implemento io. Dimmi! 🎯

4 hours ago

Your message

tutto il materiale necessario per implementare la versione Base44 completa: entità (JSON descrittivo), backend function partecipateInTask (JS), webhook partner, job timeout, seed, UI mapping e comandi di test. Non contiene chiavi segrete; imposta i segreti direttamente in Base44.

1 — Entità (descrizione JSON da creare in Base44) Copia/incolla o ricrea queste entità in Base44 (nomi, campi e tipi). I tipi indicati sono concisi: uuid, number, text, boolean, json, timestamp, int.

```
json { "entities": [ { "name": "User", "fields":  
  [ { "name": "id", "type": "uuid", "pk": true },  
    { "name": "email", "type": "text", "unique": true },  
    { "name": "balance", "type": "number", "precision": "14,2", "default": 0 },  
    { "name": "level", "type": "text" }, { "name": "kyc_status", "type": "text" },  
    { "name": "created_at", "type": "timestamp", "default": "now" } ] },  
  { "name": "Task", "fields": [ { "name": "id", "type": "uuid", "pk": true },  
    { "name": "title", "type": "text" },  
    { "name": "required_quota", "type": "number", "precision": "14,2", "default": 0 },  
    { "name": "total_reward", "type": "number", "precision": "14,2", "default": 0 },  
    { "name": "cofinancing_allowed", "type": "boolean", "default": false },  
    { "name": "partner_id", "type": "uuid" },  
    { "name": "created_at", "type": "timestamp", "default": "now" } ] },  
  { "name": "UserTask", "fields":  
    [ { "name": "id", "type": "uuid", "pk": true },  
      { "name": "user_id", "type": "uuid", "fk": "User.id" },  
      { "name": "task_id", "type": "uuid", "fk": "Task.id" },  
      { "name": "status", "type": "text" },  
      { "name": "user_quota", "type": "number", "precision": "14,2", "default": 0 },  
      { "name": "fr_quota", "type": "number", "precision": "14,2", "default": 0 },  
      { "name": "idempotency_key", "type": "text", "unique": true, "nullable": true },  
      { "name": "created_at", "type": "timestamp", "default": "now" },  
      { "name": "updated_at", "type": "timestamp", "default": "now" } ] },  
  { "name": "Cofinancing", "fields":  
    [ { "name": "id", "type": "uuid", "pk": true },  
      { "name": "user_task_id", "type": "uuid", "fk": "UserTask.id" },  
      { "name": "fr_amount", "type": "number", "precision": "14,2", "default": 0 },  
      { "name": "user_amount", "type": "number", "precision": "14,2", "default": 0 }
```

```
:0}, {"name":"status","type":"text","default":"pending"},
{"name":"created_at","type":"timestamp","default":"now"} ] },
{ "name": "Transaction", "fields":
[ {"name":"id","type":"uuid","pk":true},
{"name":"user_id","type":"uuid","nullable":true,"fk":"User.id"},
{"name":"amount","type":"number","precision":"14,2"},
{"name":"type","type":"text"},
{"name":"reference_id","type":"uuid"},
{"name":"status","type":"text","default":"pending"},
{"name":"created_at","type":"timestamp","default":"now"} ] },
{ "name": "FRLiquidity", "fields":
[ {"name":"id","type":"int","pk":true,"default":1},
{"name":"available_amount","type":"number","precision":"18,2","def
ault":0},
{"name":"updated_at","type":"timestamp","default":"now"} ] },
{ "name": "AuditLog", "fields":
[ {"name":"id","type":"uuid","pk":true},
{"name":"actor","type":"text"}, {"name":"action","type":"text"},
{"name":"payload","type":"json"},
{"name":"created_at","type":"timestamp","default":"now"} ] } ] } 2 —
```

Backend function participateInTask (JS, paste in Base44 server function) Questa funzione implementa la logica atomica: idempotency, controllo saldo, riserva utente, riserva FR, inserimenti user_task, transactions, cofinancing, audit. Adatta le chiamate DB alla sintassi SDK di Base44 (qui pseudocodice JS pronto).

```
js // participateInTask(payload) // payload: { user_id, task_id,
request_cofinancing (bool), idempotency_key (optional) } // Return:
{ status:'success'|'error', data: {...} }
```

```
async function participateInTask(ctx, payload) { const db = ctx.db; //
Base44 DB SDK handle (adatta al tuo runtime) const { user_id,
task_id, request_cofinancing = false, idempotency_key = null } =
payload;
```

```
// 1) Idempotency check if (idempotency_key) { const existing = await
db.UserTask.findOne({ where: { idempotency_key } }); if (existing)
{ const tx = await db.Transaction.findOne({ where: { reference_id:
existing.id } }); return { status: 'success', data: { task_id:
existing.task_id, user_balance_deducted: existing.user_quota,
fr_cofinancing_amount: existing.fr_quota, status: existing.status,
tx_id: tx ? tx.id : null, user_task_id: existing.id } }; } }
```

```
// 2) Read user and task (for update semantics emulate with
transaction) return await db.transaction(async (trx) => { const v_user
= await trx.User.findOne({ where: { id: user_id }, lock: true }); if (!
v_user) return { status: 'error', error_code: 'USER_NOT_FOUND',
message: 'User not found' };
```

```
const v_task = await trx.Task.findOne({ where: { id: task_id } });
if (!v_task) return { status: 'error', error_code: 'TASK_NOT_FOUND',
message: 'Task not found' };

let user_quota = 0, fr_quota = 0, cofinancing_active = false;

if (Number(v_user.balance) >= Number(v_task.required_quota)) {
  user_quota = Number(v_task.required_quota);
  fr_quota = 0;
} else if (request_cofinancing && v_task.cofinancing_allowed &&
Number(v_user.balance) >= (Number(v_task.required_quota) / 2)) {
  user_quota = Number(v_task.required_quota) / 2;
  fr_quota = Number(v_task.required_quota) / 2;
  cofinancing_active = true;

  const fr = await trx.FRLiquidity.findOne({ where: { id: 1 }, lock:
true });
  if (!fr || Number(fr.available_amount) < fr_quota) {
    return { status: 'error', error_code: 'FR_INSUFFICIENT_LIQUIDITY',
message: 'FairReward cannot cover cofinancing now' };
  }
  // reserve FR funds
  await trx.FRLiquidity.update({ available_amount:
Number(fr.available_amount) - fr_quota, updated_at: new Date() }, { where:
{ id: 1 } });
} else {
  return { status: 'error', error_code: 'INSUFFICIENT_FUNDS', message:
'Saldo insufficiente o co-finanziamento non applicabile' };
}

// 3) Deduct user balance
await trx.User.update({ balance: Number(v_user.balance) - user_quota },
{ where: { id: user_id } });

// 4) Create user_task
const user_task_id = db.uuid(); // generate uuid via SDK
await trx.UserTask.create({
  id: user_task_id,
  user_id,
  task_id,
  status: 'in_progress',
  user_quota,
  fr_quota,
  idempotency_key,
  created_at: new Date(),
  updated_at: new Date()
});

// 5) Create transactions
const tx_id = db.uuid();
await trx.Transaction.create({
```



```

    id: tx_id,
    user_id,
    amount: -user_quota,
    type: 'reserve_for_task',
    reference_id: user_task_id,
    status: 'pending',
    created_at: new Date()
  });

  if (cofinancing_active) {
    await trx.Cofinancing.create({
      id: db.uuid(),
      user_task_id,
      fr_amount: fr_quota,
      user_amount: user_quota,
      status: 'pending',
      created_at: new Date()
    });
    // FR reserve transaction (user_id null)
    await trx.Transaction.create({
      id: db.uuid(),
      user_id: null,
      amount: -fr_quota,
      type: 'fr_reserve',
      reference_id: user_task_id,
      status: 'pending',
      created_at: new Date()
    });
  }

  // 6) Audit
  await trx.AuditLog.create({
    id: db.uuid(),
    actor: 'system',
    action: 'participate_in_task',
    payload: { user_id, task_id, user_quota, fr_quota, idempotency_key },
    created_at: new Date()
  });

  return {
    status: 'success',
    data: {
      task_id,
      user_balance_deducted: user_quota,
      fr_cofinancing_amount: fr_quota,
      status: 'in_progress',
      tx_id,
      user_task_id
    }
  };
}); } Nota: adatta db, trx, db.uuid() e transaction alla SDK/DSL di
Base44. La logica transazionale è già completa: idempotency, lock/
read, update, inserts, audit.

```

3 — Webhook partner e job timeout (server functions) Webhook partner (JS) — endpoint partnerWebhookTaskCompleted:

```
js // payload: { user_task_id, success (bool), partner_reference } //  
header: HMAC signature checked against WEBHOOK_SECRET
```

```
async function partnerWebhookTaskCompleted(ctx, payload,  
headers) { const db = ctx.db; const secret =  
ctx.env.WEBHOOK_SECRET; // verify HMAC (implement per Base44  
runtime) if (!verifyHmac(headers['x-signature'],  
JSON.stringify(payload), secret)) { return { status: 'error', error_code:  
'UNAUTHORIZED' }; }
```

```
return await db.transaction(async (trx) => { const ut = await  
trx.UserTask.findOne({ where: { id: payload.user_task_id }, lock:  
true }); if (!ut) return { status: 'error', error_code:  
'USER_TASK_NOT_FOUND' };
```

```
if (payload.success) {  
  // mark completed  
  await trx.UserTask.update({ status: 'completed', updated_at: new  
Date() }, { where: { id: ut.id } });  
  // finalize transactions: mark reserve pending -> completed, create  
payout  
  await trx.Transaction.update({ status: 'completed' }, { where:  
{ reference_id: ut.id } });  
  // create payout transaction for user reward  
  await trx.Transaction.create({  
    id: db.uuid(),  
    user_id: ut.user_id,  
    amount: ut.fr_quota + ut.user_quota, // or total_reward logic  
    type: 'payout',  
    reference_id: ut.id,  
    status: 'completed',  
    created_at: new Date()  
  });  
  // mark cofinancing completed if exists  
  await trx.Cofinancing.update({ status: 'completed' }, { where:  
{ user_task_id: ut.id } });  
  await trx.AuditLog.create({ id: db.uuid(), actor: 'partner', action:  
'task_completed', payload, created_at: new Date() });  
  return { status: 'success' };  
} else {  
  // failure -> refund user and restore FR liquidity if used  
  await trx.User.update({ balance: db.sequelize.literal(`balance + $  
{ut.user_quota}`) }, { where: { id: ut.user_id } });  
  await trx.Transaction.create({ id: db.uuid(), user_id: ut.user_id,  
amount: ut.user_quota, type: 'refund', reference_id: ut.id, status:  
'completed', created_at: new Date() });  
  if (ut.fr_quota > 0) {  
    await trx.FRLiquidity.update({ available_amount:  
db.sequelize.literal(`available_amount + ${ut.fr_quota}`), updated_at: new  
Date() }, { where: { id: 1 } });  
    await trx.Transaction.create({ id: db.uuid(), user_id: null, amount:  
ut.fr_quota, type: 'fr_refund', reference_id: ut.id, status: 'completed',  
created_at: new Date() });  
    await trx.Cofinancing.update({ status: 'cancelled' }, { where:
```

```

{ user_task_id: ut.id } });
}
await trx.UserTask.update({ status: 'cancelled', updated_at: new
Date() }, { where: { id: ut.id } });
await trx.AuditLog.create({ id: db.uuid(), actor: 'partner', action:
'task_failed', payload, created_at: new Date() });
return { status: 'success' };
}
}); } Job timeout (scheduled function) — pseudocodice:

```

```

js // run every 5-15 minutes async function timeoutJob(ctx) { const db
= ctx.db; const thresholdMinutes =
Number(ctx.env.TIMEOUT_MINUTES || 60); const cutoff = new
Date(Date.now() - thresholdMinutes * 60000);

```

```

const toTimeout = await db.UserTask.findAll({ where: { status:
'in_progress', created_at: { $lt: cutoff } } }); for (const ut of
toTimeout) { await db.transaction(async (trx) => { // refund user await
trx.User.update({ balance: trx.sequelize.literal(balance + $
{ut.user_quota} ) }, { where: { id: ut.user_id } }); await
trx.Transaction.create({ id: db.uuid(), user_id: ut.user_id, amount:
ut.user_quota, type: 'refund_timeout', reference_id: ut.id, status:
'completed', created_at: new Date() }); // restore FR liquidity if used if
(ut.fr_quota > 0) { await trx.FRLiquidity.update({ available_amount:
trx.sequelize.literal(available_amount + ${ut.fr_quota}),
updated_at: new Date() }, { where: { id: 1 } }); await
trx.Transaction.create({ id: db.uuid(), user_id: null, amount:
ut.fr_quota, type: 'fr_refund_timeout', reference_id: ut.id, status:
'completed', created_at: new Date() }); await
trx.Cofinancing.update({ status: 'cancelled' }, { where: { user_task_id:
ut.id } }); } await trx.UserTask.update({ status: 'timed_out',
updated_at: new Date() }, { where: { id: ut.id } }); await
trx.AuditLog.create({ id: db.uuid(), actor: 'system', action:
'timeout_refund', payload: { user_task_id: ut.id }, created_at: new
Date() }); }); } } Nota: adatta sequelize.literal o equivalente al DSL di
Base44.

```

4 — Seed dati (SQL / pseudo-SDK) e comandi di test Seed (esegui in DB Base44 o via SDK):

```

sql -- create demo user INSERT INTO users (id, email, balance,
created_at) VALUES (gen_random_uuid(), 'tester@example.com',
10.00, now());

```

```

-- create demo task INSERT INTO tasks (id, title, required_quota,
total_reward, cofinancing_allowed, created_at) VALUES

```

```
(gen_random_uuid(), 'Download Base', 10.00, 24.00, true, now());  
Comandi curl di test (adatta URL e token Base44):
```

bash

happy path (user covers full quota)

```
curl -X POST "https://<base44>/functions/participateInTask"  
-H "Content-Type: application/json"  
-H "Authorization: Bearer <BASE44_TEST_TOKEN>"  
-H "Idempotency-Key: test-happy-1"  
-d  
'{"user_id":"<USER_ID>","request_cofinancing":false,"task_id":"<TASK_ID>"}'
```

cofinancing path (user has half balance)

```
curl -X POST "https://<base44>/functions/participateInTask"  
-H "Content-Type: application/json"  
-H "Authorization: Bearer <BASE44_TEST_TOKEN>"  
-H "Idempotency-Key: test-cofin-1"  
-d  
'{"user_id":"<USER_ID>","request_cofinancing":true,"task_id":"<TASK_ID>"}'
```

Test checklist (esegui e registra risultati):

Happy path, Cofinancing path, Insufficient funds, FR liquidity exhausted, Idempotency (same idempotency key twice), Webhook success, Timeout refund.

5 — UI mapping (componenti Base44 e binding) Crea queste pagine/components in Base44 e collega ai dati:

Home

Data: currentUser.balance

List: Task (required_quota, total_reward, cofinancing_allowed)

Action: Inizia ora → apre Modal Conferma

Modal Conferma

Inputs: mostra required_quota, calcola user_quota e fr_quota client-side

Checkbox: accetto prelievo

Confirm: chiama participateInTask backend function con { user_id: currentUser.id, task_id, request_cofinancing: true/false,

idempotency_key: uuid() }

On success: aggiorna currentUser.balance (sottrai user_balance_deducted), mostra toast

Il mio lavoro

List: UserTask filtered by user_id

Show status, created_at, amounts

Cassa

List: Transaction filtered by user_id

Profilo

Show User fields, logout

Error handling

Map server error codes to user messages:

INSUFFICIENT_FUNDS → "Saldo insufficiente"

FR_INSUFFICIENT_LIQUIDITY → "FairReward non può cofinanziare ora"

USER_NOT_FOUND / TASK_NOT_FOUND → "Errore interno"

6 — Variabili d'ambiente e segreti (impostare in Base44)

WEBHOOK_SECRET — HMAC secret per webhook partner

TIMEOUT_MINUTES — soglia timeout (es. 60)

BASE44_TEST_TOKEN — token per test (non in produzione)

7 — Consegna finale e file che ti fornisco subito Ho preparato tutto:

Entità JSON (sezione 1)

Backend function JS (sezione 2)

Webhook + timeout (sezione 3)

Seed + curl tests (sezione 4)

UI mapping (sezione 5)

Env list (sezione 6)

Se vuoi, ora procedo a:

A. generare i file pronti (zip) con i codici adattati a un runtime Node/Express (se preferisci esportare), oppure

B. lasciarti questo pacchetto testuale da incollare in Base44 (consigliato).

Ho seguito la tua richiesta: niente Supabase/Netlify, tutto pronto per Base44. Ho evitato di includere chiavi sensibili. Se vuoi che incolli ora il codice già adattato a uno specifico editor Base44 (o che generi i singoli file in ordine), scrivi "Incolla file" e procedo subito con i singoli file completi.

Show more

3 hours ago

Editing isn't available while the app is building

Revert

Reverting isn't available while the app is building

Base44 response

Procedo a costruire l'intera app FairReward su Base44. Prima controllo il formato delle backend function e dei task schedulati sulla piattaforma, poi costruisco tutto in parallelo.

Checked the docs

Base44 response

Ho il formato. Ora costruisco tutto: entità, backend function, pagine e seed. Parto con entità + tema + check dei componenti auth esistenti.

Updated the Task, User Task e 7 more

Base44 response

Tema e entità pronti. Ora scrivo le 4 backend function, i componenti, le pagine e collego il router.

Updated the Layout, Participate Modal e 6 more

Base44 response

Le backend function richiedono il piano Builder+ e non posso crearle sul piano attuale. Implemento quindi la logica di partecipazione lato client (usando l'SDK delle entità, accessibile dal frontend) così l'app funziona comunque end-to-end, e lascio il codice pronto da spostare in una backend function quando si passa a Builder+.

Updated the participate, Home Page e 3 more

Base44 response

Aggiungo le route di autenticazione (login/register/forgot/reset) che mancano nel router, così gli utenti non autenticati vengono correttamente indirizzati.

Updated App

Base44 response

Verifico lo stato finale del router.

Read App

Base44 response

Thought for 7s

Fatto. Ho costruito l'intera app FairReward 50/50.

Cosa c'è ora (funzionante):

- **7 entità:** Task, UserTask, Cofinancing, Transaction, FRLiquidity, AuditLog + campi custom su User (balance, level, kyc_status)
- **Seed:** liquidità FairReward €3.000 + task demo "Download & Registrazione Base" (quota €10, reward €24, 50/50 attivo)
- **4 pagine** con navigazione inferiore e tema dark 50/50: Home (saldo + catalogo + modale conferma), Il mio lavoro (storico partecipazioni + "Completa demo"), Cassa (transazioni), Profilo (logout)
- **Logica completa** in `src/lib/participate.js`: partecipazione con idempotency, controllo saldo, cofinanziamento 50/50, riserva liquidità FR, transazioni, audit, e webhook completamento/refund

⚠ Limitazione importante — serve Builder+: Le backend

function (`participate-in-task`, `partner-webhook-task-completed`, `timeout-job`, `grant-demo-balance`) non ho potuto crearle perché richiedono il piano Builder+. Ho quindi implementato la logica **lato client** (tramite l'SDK delle entità) così l'app è subito testabile end-to-end. Il codice è già strutturato per essere spostato in backend function quando si passa a Builder+.

Da fare quando passi a Builder+ / per la produzione:

- 1 Sposta `participate.js` nelle 3 backend function con `base44.asServiceRole` (atomicità lato server)
- 2 Aggiungi verifica HMAC sul webhook + secret `WEBHOOK_SECRET`

- 3 Crea il workflow schedulato `timeout-job` (Dashboard → Workflows, ogni 5 min)
- 4 Configura **RLS** sulle entità (oggi ogni utente autenticato vede/modifica tutti i record — insicuro per un'app finanziaria)
- 5 Imposta i secret `WEBHOOK_SECRET` e `TIMEOUT_MINUTES`

Per testare ora: registrati → su Home premi "Richiedi saldo demo (€10)" → "Inizia ora" sul task → "Partecipa" → vai su "Il mio lavoro" e premi "Completa (demo)" per vedere la ricompensa accreditata in Cassa.

3 hours ago

Your message

Interfaccia non mi piace per niente deve essere più chiara le cose importa di vanno messe più grandi e risalto ma non con i colori del circo ma devono risaltare il fatto che noi non chiediamo soldi ma facciamo usare su dati i soldi guadagnati il fatto che possono scegliere se lavorare in base al tempo che anno base a quanti soldi hanno bisogno e noi gli diciamo sempre se tu hai bisogno di 10 € devi fare queste tasche queste queste questo però ci metti troppo tempo cioè non così non si capisce niente non non va bene per quello che ti dicevo facciamo Cosa ti consegno ora (tutto pronto per Base44 senza segreti) Entità (User, Task, UserTask, Cofinancing, Transaction, FRLiquidity, AuditLog) — JSON già fornito.

Client-side function `participateInTaskClient` (JS) che replica la logica transazionale lato client usando le entità Base44 SDK; gestisce idempotency, riserve, cofinanziamento e scrive audit.

Webhook simulator `partnerWebhookTaskCompletedClient` per testare completamento/refund (simula il comportamento del partner e applica refund/payout).

Timeout job simulator `timeoutJobClient` eseguibile da un admin o scheduler esterno per simulare il job di timeout.

Grant demo balance funzione client per dare saldo demo (€10) ai tester.

Istruzioni di migrazione per spostare ogni funzione in backend function Builder+ (HMAC webhook, atomicità, RLS).

Test commands curl e checklist già forniti.

Implementazioni client-side pronte da incollare (adatta al tuo runtime Base44 SDK) Copia/incolla questi snippet nelle serverless scripts o in uno script admin che esegui da console Base44. Sono scritte in JS pseudoreale, adattale al tuo SDK (db.* → chiamate Base44).

```
participateInTaskClient.js js // payload: { user_id, task_id,
request_cofinancing, idempotency_key } // Requires: db SDK with
transaction support; use as admin action for tests. async function
participateInTaskClient(db, payload) { const { user_id, task_id,
request_cofinancing = false, idempotency_key = null } = payload;
```

```
// Idempotency check if (idempotency_key) { const existing = await
db.UserTask.findOne({ where: { idempotency_key } }); if (existing)
{ const tx = await db.Transaction.findOne({ where: { reference_id:
existing.id } }); return { status: 'success', data: { task_id:
existing.task_id, user_balance_deducted: existing.user_quota,
fr_cofinancing_amount: existing.fr_quota, status: existing.status,
tx_id: tx ? tx.id : null, user_task_id: existing.id } }; }
```

```
// Emulate atomicity with DB transaction (if SDK supports) return
await db.transaction(async (trx) => { const user = await
trx.User.findOne({ where: { id: user_id }, lock: true }); if (!user) return
{ status: 'error', error_code: 'USER_NOT_FOUND' };
```

```
const task = await trx.Task.findOne({ where: { id: task_id } });
if (!task) return { status: 'error', error_code: 'TASK_NOT_FOUND' };

let user_quota = 0, fr_quota = 0, cofinancing_active = false;

if (Number(user.balance) >= Number(task.required_quota)) {
  user_quota = Number(task.required_quota);
} else if (request_cofinancing && task.cofinancing_allowed &&
Number(user.balance) >= (Number(task.required_quota) / 2)) {
  user_quota = Number(task.required_quota) / 2;
  fr_quota = Number(task.required_quota) / 2;
  cofinancing_active = true;

  const fr = await trx.FRLiquidity.findOne({ where: { id: 1 }, lock:
true });
  if (!fr || Number(fr.available_amount) < fr_quota) {
    return { status: 'error', error_code: 'FR_INSUFFICIENT_LIQUIDITY' };
  }
  await trx.FRLiquidity.update({ available_amount:
Number(fr.available_amount) - fr_quota, updated_at: new Date() }, { where:
{ id: 1 } });
} else {
  return { status: 'error', error_code: 'INSUFFICIENT_FUNDS' };
}
```

```

await trx.User.update({ balance: Number(user.balance) - user_quota },
{ where: { id: user_id } });

const user_task_id = db.uuid();
await trx.UserTask.create({ id: user_task_id, user_id, task_id, status:
'in_progress', user_quota, fr_quota, idempotency_key, created_at: new
Date(), updated_at: new Date() });

const tx_id = db.uuid();
await trx.Transaction.create({ id: tx_id, user_id, amount: -user_quota,
type: 'reserve_for_task', reference_id: user_task_id, status: 'pending',
created_at: new Date() });

if (cofinancing_active) {
  await trx.Cofinancing.create({ id: db.uuid(), user_task_id, fr_amount:
fr_quota, user_amount: user_quota, status: 'pending', created_at: new
Date() });
  await trx.Transaction.create({ id: db.uuid(), user_id: null, amount:
-fr_quota, type: 'fr_reserve', reference_id: user_task_id, status:
'pending', created_at: new Date() });
}

await trx.AuditLog.create({ id: db.uuid(), actor: 'client', action:
'participate_in_task', payload: { user_id, task_id, user_quota, fr_quota,
idempotency_key }, created_at: new Date() });

return { status: 'success', data: { task_id, user_balance_deducted:
user_quota, fr_cofinancing_amount: fr_quota, status: 'in_progress', tx_id,
user_task_id } };
}); } partnerWebhookTaskCompletedClient.js (simulatore) js //
payload: { user_task_id, success, partner_reference } // Use as admin
action to simulate partner callback async function
partnerWebhookTaskCompletedClient(db, payload) { return await
db.transaction(async (trx) => { const ut = await
trx.UserTask.findOne({ where: { id: payload.user_task_id }, lock:
true }); if (!ut) return { status: 'error', error_code:
'USER_TASK_NOT_FOUND' };

```

```

if (payload.success) {
  await trx.UserTask.update({ status: 'completed', updated_at: new
Date() }, { where: { id: ut.id } });
  await trx.Transaction.update({ status: 'completed' }, { where:
{ reference_id: ut.id } });
  await trx.Transaction.create({ id: db.uuid(), user_id: ut.user_id,
amount: (ut.user_quota + ut.fr_quota), type: 'payout', reference_id:
ut.id, status: 'completed', created_at: new Date() });
  await trx.Cofinancing.update({ status: 'completed' }, { where:
{ user_task_id: ut.id } });
  await trx.AuditLog.create({ id: db.uuid(), actor: 'partner', action:
'task_completed', payload, created_at: new Date() });
  return { status: 'success' };
} else {
  await trx.User.update({ balance: Number((await trx.User.findOne({ where:
{ id: ut.user_id } })).balance) + ut.user_quota }, { where: { id:
ut.user_id } });
  await trx.Transaction.create({ id: db.uuid(), user_id: ut.user_id,

```

```

amount: ut.user_quota, type: 'refund', reference_id: ut.id, status:
'completed', created_at: new Date() });
  if (ut.fr_quota > 0) {
    await trx.FRLiquidity.update({ available_amount: Number((await
trx.FRLiquidity.findOne({ where: { id: 1 } })).available_amount) +
ut.fr_quota, updated_at: new Date() }, { where: { id: 1 } });
    await trx.Transaction.create({ id: db.uuid(), user_id: null, amount:
ut.fr_quota, type: 'fr_refund', reference_id: ut.id, status: 'completed',
created_at: new Date() });
    await trx.Cofinancing.update({ status: 'cancelled' }, { where:
{ user_task_id: ut.id } });
  }
  await trx.UserTask.update({ status: 'cancelled', updated_at: new
Date() }, { where: { id: ut.id } });
  await trx.AuditLog.create({ id: db.uuid(), actor: 'partner', action:
'task_failed', payload, created_at: new Date() });
  return { status: 'success' };
}

```

```

}); } timeoutJobClient.js (simulatore) js // run manually or via external
scheduler until you have Builder+ scheduled workflows async
function timeoutJobClient(db, minutesThreshold = 60) { const cutoff
= new Date(Date.now() - minutesThreshold * 60000); const
toTimeout = await db.UserTask.findAll({ where: { status:
'in_progress', created_at: { $lt: cutoff } } }); for (const ut of
toTimeout) { await db.transaction(async (trx) => { await
trx.User.update({ balance: Number((await trx.User.findOne({ where:
{ id: ut.user_id } })).balance) + ut.user_quota }, { where: { id:
ut.user_id } }); await trx.Transaction.create({ id: db.uuid(), user_id:
ut.user_id, amount: ut.user_quota, type: 'refund_timeout',
reference_id: ut.id, status: 'completed', created_at: new Date() }); if
(ut.fr_quota > 0) { await trx.FRLiquidity.update({ available_amount:
Number((await trx.FRLiquidity.findOne({ where: { id:
1 } })).available_amount) + ut.fr_quota, updated_at: new Date() },
{ where: { id: 1 } }); await trx.Transaction.create({ id: db.uuid(),
user_id: null, amount: ut.fr_quota, type: 'fr_refund_timeout',
reference_id: ut.id, status: 'completed', created_at: new Date() });
await trx.Cofinancing.update({ status: 'cancelled' }, { where:
{ user_task_id: ut.id } }); } await trx.UserTask.update({ status:
'timed_out', updated_at: new Date() }, { where: { id: ut.id } }); await
trx.AuditLog.create({ id: db.uuid(), actor: 'system', action:
'timeout_refund', payload: { user_task_id: ut.id }, created_at: new
Date() }); }); } return { status: 'ok', processed: toTimeout.length }; }
grantDemoBalanceClient.js js // admin action to give demo balance to
current user async function grantDemoBalanceClient(db, user_id,
amount = 10.00) { await db.User.update({ balance: Number((await
db.User.findOne({ where: { id: user_id } })).balance) + amount },
{ where: { id: user_id } }); await db.Transaction.create({ id: db.uuid(),

```

```
user_id, amount, type: 'demo_grant', reference_id: null, status:
'completed', created_at: new Date() }); await db.AuditLog.create({ id:
db.uuid(), actor: 'system', action: 'grant_demo_balance', payload:
{ user_id, amount }, created_at: new Date() }); return { status:
'success' }; } Note operative e sicurezza (essenziali) Queste
implementazioni client-side funzionano per test ma non sono sicure
per produzione: un utente malintenzionato con accesso SDK/admin
potrebbe manipolare operazioni.
```

Quando passi a Builder+ sposta immediatamente `participateInTask`, `partnerWebhookTaskCompleted`, `timeoutJob` e `grantDemoBalance` in backend functions con service-role privileges e HMAC verification per webhook.

RLS: configura Row Level Security sulle entità (`UserTask`, `Transaction`, `Cofinancing`) prima di andare in produzione.

Secrets: imposta `WEBHOOK_SECRET` e `TIMEOUT_MINUTES` in Base44 env; non inserirli nei file.

Migrazione a Builder+ (passaggi sintetici) Crea backend functions
Builder+: `participateInTask`, `partnerWebhookTaskCompleted`, `timeoutJob`, `grantDemoBalance`.

Copia il codice server (adatta i `db.transaction` e rimuovi qualsiasi client-side exposure).

Abilita HMAC per webhook e verifica signature in `partnerWebhookTaskCompleted`.

Crea scheduled workflow per `timeoutJob` (ogni 5–15 min).

Abilita RLS e policy per limitare accessi.

Rimuovi logica sensibile dal client e usa solo chiamate a backend functions.

Test rapido che puoi eseguire ora Crea demo user e demo task (seed).

Esegui `grantDemoBalanceClient(db, <USER_ID>)`.

Chiama `participateInTaskClient(db, { user_id, task_id, request_cofinancing: true, idempotency_key: 'test-1' })`.

Simula partner con `partnerWebhookTaskCompletedClient(db,`

```
{ user_task_id, success: true }).
```

Esegui `timeoutJobClient(db, 60)` per simulare timeout refund.

Consegna finale Hai ora tutti i pezzi per far partire e testare l'app su Base44 senza Builder+. Quando passi a Builder+ ti fornisco la versione server-ready (stessa logica, con HMAC e policy) pronta da incollare nelle backend function.

Ho preparato il codice client-side, il simulatore webhook, il job timeout e le istruzioni di migrazione: tutto pronto. Se vuoi, incollami qui il nome esatto del tuo progetto Base44 e confermo i nomi delle entità così te li restituisco in formato JSON pronto per import.

Show more

3 hours ago

Editing isn't available while the app is building

Revert

Reverting isn't available while the app is building

Base44 response

Ridisegno l'interfaccia: tipografia più grande e chiara, accento più sobrio (niente "colori del circo"), messaggio "non chiediamo soldi, usi ciò che guadagni" in evidenza, e aggiungo il **pianificatore** ("di quanto hai bisogno? → ecco i task e il tempo totale"). Aggiungo anche il campo tempo stimato ai task.

Updated the Task, styles e 3 more

Approval Required

Imposta tempo stimato 3 min sul task demo

ApproveReject

3 hours ago

Suggestions

1 message queued

Aggiungi dei filtri nella pagina Home per permettere di ordinare i task in base alla quota richiesta o al reward totale. Serve a rendere più veloce la ricerca delle opportunità di guadagno migliori.

Auto

Discuss

Home

Upgrade